

GUESSTHEWORD.

PROGETTAZIONE E ARCHITETTURA DEL SISTEMA

Programmazione Java Avanzata - A.A. 2025/26

Gruppo 7 E. Ferrentino R. Genovese L. Canzolino F. P. De Piano



ARCHITETTURA DI SISTEMA

DISTRIBUTED CLIENT-SERVER

Il sistema comunica in tempo reale su rete **TCP/IP**. L'intera architettura è stata divisa fisicamente in tre moduli per garantire un basso accoppiamento e alta coesione:

- ◆ **Modulo Server:** Nodo autoritativo centrale che orchestra l'autenticazione, la concorrenza e le sfide.
- ◆ **Modulo Client:** Interfaccia leggera ed asincrona delegata solo a catturare l'input.
- ◆ **Modulo Common:** Condivisione dei DTO essenziali tra Client e Server.

PATTERN MODEL-VIEW-CONTROLLER

L'approccio MVC isola nettamente la logica di presentazione dal codice sorgente:

- ◆ **View:** Realizzata integralmente in JavaFX mediante file XML e l'utilizzo di SceneBuilder.
- ◆ **Controller:** Classi Java dedicate alla gestione degli eventi grafici e all'interazione con livello di servizio o di rete.
- ◆ **Model:** Classi di dominio che conservano lo stato dell'applicazione.

PERSISTENZA E PATTERN DAO

GESTIONE DATI SINCRONA E SICURA

Per la persistenza delle partite e delle credenziali è stato impiegato **SQLite**, motore DBMS leggero e integrato privo di configurazione esterna complessa.

- ✦ **Pattern DAO (Data Access Object):** Astrazione completa fornita dalle interfacce (UtenteDAO, PartitaDAO, AdminDAO(Server)).
- ✦ **Disaccoppiamento SQL:** Il livello logico ed i gestori (es. GestoreAutenticazione) ignorano le specifiche query.
- ✦ **Scalabilità Aperta:** La separazione permette la migrazione del backend ad altro DBMS senza intaccare il core.
- ✦ **Scelta ID Utente:** Chiave surrogata numerica anziché username per facilitare evoluzioni future del sistema.





SCELTE PROGETTUALI PRINCIPALI



SOCKET & SERIALIZZAZIONE

Utilizzo di Socket TCP per garantire l'integrità del traffico di rete. Scambio dati tramite serializzazione diretta di oggetti Java incapsulati nella classe **Messaggio**.



MULTITHREADING & CONCORRENZA

L'architettura Server implementa il modello **Thread-per-Client**. Ogni socket delega ad un ClientHandler isolato in Runnable per gestire I/O non bloccante.



TASK UI NON BLOCCANTE

Uso integrato delle classi Task e Service in JavaFX. L'analisi del testo non produce alcun fenomeno di freezing visivo sulle finestre di amministrazione.

SICUREZZA E PREVENZIONE CHEAT

100%

SERVER AUTORITATIVO

PROTOCOLLO E CAMPI TRANSIENT

Nelle architetture distribuite la sicurezza è prioritaria. Seguendo i dettami del server autoritativo, la soluzione non risiede mai lato client.

- ♦ Nell'oggetto di rete **Sfida**, la parola da indovinare e lo shift del cifrario sono dichiarati **transient**.
- ♦ Questo impedisce la serializzazione automatica e la trasmissione sulla rete.
- ♦ Sventa completamente attacchi tramite reverse engineering o packet sniffing locale.



ANALISI DEI TESTI (STREAM API)



TERM FREQUENCY & PIPELINE FUNZIONALI

L'elaborazione asincrona del testo per il calcolo della frequenza delle parole sfrutta a pieno la potenza di Java 8:

- ◆ **Stream API:** Utilizzato per la pulizia del testo, la tokenizzazione ed il raggruppamento in mappa frequenze.
- ◆ **Prestazioni:** Massima efficienza anche su documenti di grandi dimensioni senza gravare sulla memoria heap.
- ◆ **Serializzazione:** Le analisi prodotte possono essere persistite in file binari .dat per essere utilizzate all'avvio.

MODELLO DEI DATI DEL DATABASE

Entità ER	Descrizione Funzionale	Attributi Chiave / Chiavi Esterne
Utente	Infrastruttura di accesso e credenziali degli account	ID_Utente (PK), Username (Unique), Password, Stato
Admin	Utente con permessi avanzati di analisi ed amministrazione	ID_Utente (FK -> Utente)
Giocatore	Utente ordinario partecipante alla coda di gioco online	ID_Utente (FK -> Utente)
Partita	Sessione concorrente o passata tra due giocatori distinti	ID_Partita (PK), ID_Avversario1 (FK -> Giocatore), ID_Avversario2 (FK -> Giocatore), Durata, Esito, Vincitore(FK -> Giocatore)



MODELLI DELLE CLASSI CONDIVISE

MODELLO CONDIVISO (COMMON)

Contiene i contratti e le definizioni scambiate continuamente tra client e server, preservando lo stesso serializzazione ID:

- ◆ Utente, Giocatore, Partita: Strutture dati essenziali.
- ◆ Admin: Presente unicamente a livello del modulo Server.
- ◆ L'ereditarietà è definita per mantenere l'interfaccia leggera.

PROTOCOLLO DI COMUNICAZIONE

Lo scambio dati asincrono si fonda su messaggi fortemente tipizzati:

- ◆ Messaggio: Contiene TipoMessaggio ed un payload astratto.
- ◆ Sfida: Testo oscurato ed informazioni di durata trasmesse al client.
- ◆ EsitoSfida: Informazioni sul vincitore, la soluzione ed il tempo trascorso.

CICLO DEL GIOCO (MATCHMAKING)



GRAZIE PER L'ATTENZIONE

Gruppo 7 • Università degli Studi di Salerno • A.A. 2025/26